

Exercise 7: Financial Forecasting

Scenario: You are developing a financial forecasting tool that predicts future values based on past data.

Step1: Understand Recursive Algorithms:

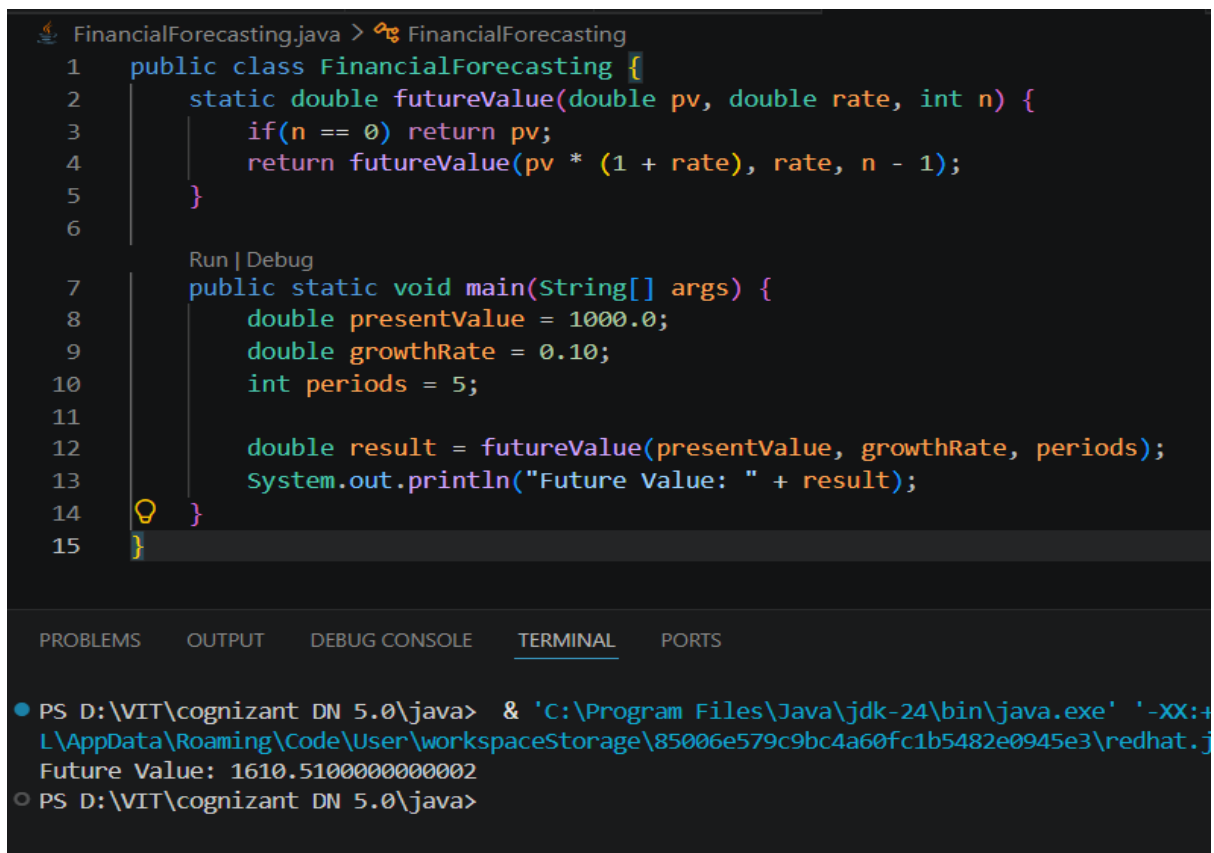
Recursion is a programming technique in which a method calls itself repeatedly until it reaches a stopping condition known as the **base case**.

A recursive algorithm consists of two parts:

- **Base Case:** Stops further recursive calls.
- **Recursive Case:** Calls the same method with a smaller or simpler input.

Recursion is useful for solving problems that can be divided into smaller versions of the same problem.

Step 2 & 3 (code and output):



```
FinancialForecasting.java > FinancialForecasting
1 public class FinancialForecasting {
2     static double futureValue(double pv, double rate, int n) {
3         if(n == 0) return pv;
4         return futureValue(pv * (1 + rate), rate, n - 1);
5     }
6
7     Run | Debug
8     public static void main(String[] args) {
9         double presentValue = 1000.0;
10        double growthRate = 0.10;
11        int periods = 5;
12
13        double result = futureValue(presentValue, growthRate, periods);
14        System.out.println("Future Value: " + result);
15    }
16 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● PS D:\VIT\cognizant DN 5.0\java> & 'C:\Program Files\Java\jdk-24\bin\java.exe' '-XX:+
L\AppData\Roaming\Code\User\workspaceStorage\85006e579c9bc4a60fc1b5482e0945e3\redhat.j
Future Value: 1610.5100000000002
○ PS D:\VIT\cognizant DN 5.0\java>
```

Step4: Analysis

Time Complexity: Each recursive call processes one period.

If there are **n** periods, the function performs **n** recursive calls.

Time Complexity: $O(n)$

Space Complexity: Each recursive call is stored on the call stack. Therefore,

Space Complexity = $O(n)$
